produces:

```
0: 10
1: 9
2: 8
3: 7
```

Strings

Ruby strings are sequences of characters. They normally hold printable characters, but that is not a requirement; a string can also hold binary data. Strings are instances of class String, and are often created using string literals—sequences of characters between delimiters.

Ruby has a lot of different ways to create string literals, which differ in how much processing is done on the characters in the string. One kind of processing is an *escape sequence*. An escape sequence allows you to represent data that is otherwise impossible to represent in the string. Escape sequences in Ruby start with a backslash (\).

The simplest literal in Ruby is the single-quoted string. Inside a single-quoted string, only two escape sequences are recognized. Two consecutive backslashes (\\) are replaced by a single backslash, and a backslash followed by a single quote (\\') becomes a single quote. In these cases, the escape sequence allows you to represent a character that would otherwise cause problems because the character has meaning to the Ruby parser.

```
'escape using "\\"'   # => escape using "\"
'That\'s right'       # => That's right
```

Note that the double quote inside the top string is handled normally.

Double-quoted strings support a boatload more escape sequences. The most common is probably \n, the newline character.

Double-quoted strings also support *string interpolation*. With string interpolation, you can substitute the value of any Ruby code into a string using the sequence #{ *expr* }. If the expression being evalutated is just a global variable, a class variable, or an instance variable, you can omit the braces, so #@foo, or #@@foo, or #$foo.

```
"Seconds/day: #{24 * 60 * 60}"             # => Seconds/day: 86400
"#{'Ho! ' * 3}Merry Christmas!"            # => Ho! Ho! Ho! Merry Christmas!
"The input file name is #$FILENAME"        # => The input file name is -
```

The interpolated code can be one or more statements, not just an expression—we don't recommend this, but…

```
puts  "now is #{
  def the(a)
    'the ' + a
  end
  the('time')
} for all bad coders..."
```

produces:

```
now is the time for all bad coders...
```

You will sometimes see style guides that prefer single-quotes if interpolation is not used on the grounds that they are faster. As far as we've been able to tell, there's no significant speed